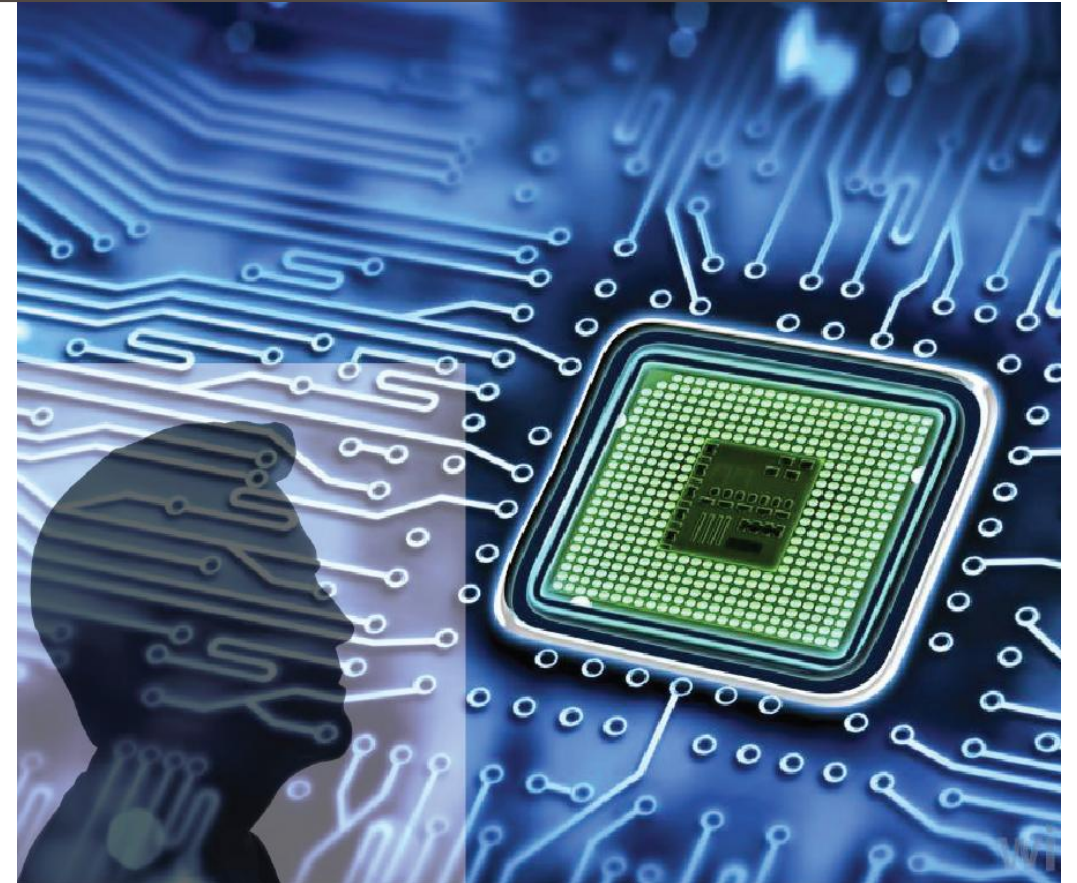
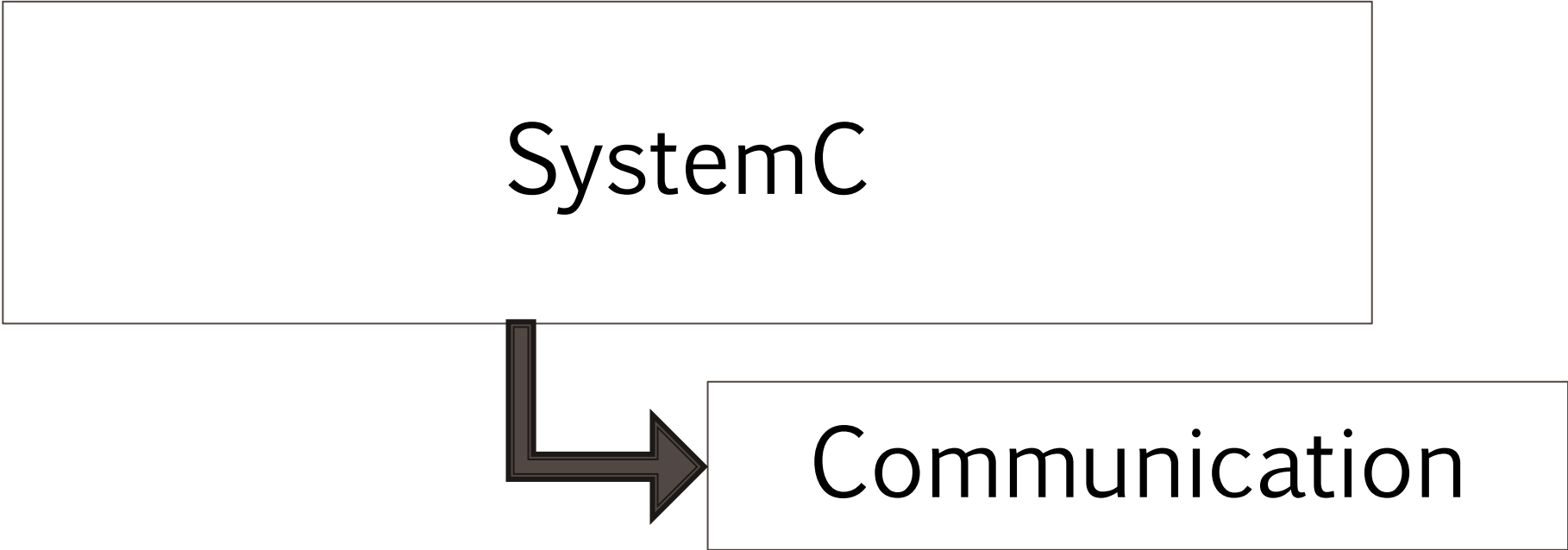


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-3, Lecture-2 SystemC

Sneh Saurabh
28th August, 2025



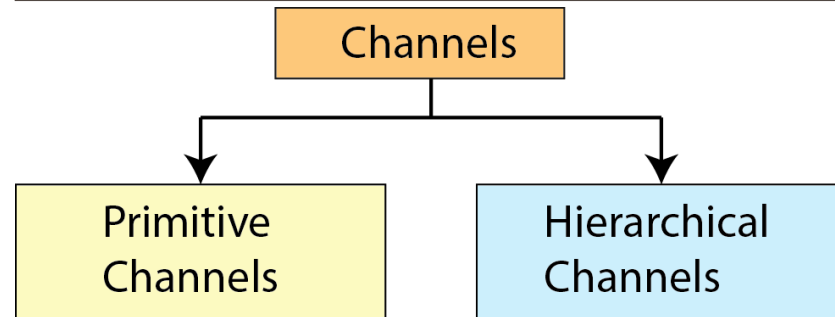


```
graph TD; A[SystemC] --> B[Communication]
```

SystemC

Communication

Channels



	Primitive Channels	Hierarchical Channels
Derived from	<code>sc_prim_channel</code>	<code>sc_channel</code> (identical to <code>sc_module</code>)
Contains ports and hierarchies	No	Yes
Contains simulation processes	No	Yes
Purpose	Simple and fast communication	Complex functionality (similar to a module with an interface implemented)
Example	<code>sc_fifo<T></code> , <code>sc_mutex</code> , <code>sc_semaphore</code>	Implementation of complex buses PCI, AMBA, etc.

Channels: sc_mutex (mutually exclusive)

- Allows multiple processes to share a common resource without colliding.

- Declare: `sc_mutex NAME`;
- `sc_mutex` channel implements `sc_mutex_if` interface class
- Member functions
 - Lock (blocking, waits if already locked): `lock()`
 - Lock (non-blocking, moves ahead if already locked): `trylock()`, returns 0 on success
 - Free a locked mutex: `unlock()`

```
SC_MODULE(ChangeRoom) {  
    sc_mutex door;  
    void customer_1_change(void); // process  
    void customer_2_change(void); // process  
    void customer_3_change(void); // process  
    SC_CTOR(ChangeRoom) {  
        SC_THREAD(customer_1_change); // register process  
        SC_THREAD(customer_2_change); // register process  
        SC_THREAD(customer_3_change); // register process  
    }  
};
```

Mutex: Example

```
SC_MODULE(ChangeRoom) {
    sc_mutex door;
    void customer_1_change(void); // process
    void customer_2_change(void); // process
    void customer_3_change(void); // process
    SC_CTOR(ChangeRoom) {
        SC_THREAD(customer_1_change); // register process
        SC_THREAD(customer_2_change); // register process
        SC_THREAD(customer_3_change); // register process
    }
};
```

```
void ChangeRoom::customer_2_change(void) {
    while (1) {
        if (door.trylock()) {
            cout << "Customer 2 wait: " << sc_time_stamp() << endl;
            door.lock();
        }
        cout << "Customer 2 start: " << sc_time_stamp() << endl;
        // carry out change
        wait(600, SC_SEC);
        cout << "Customer 2 end: " << sc_time_stamp() << endl;
        door.unlock();
        wait(SC_ZERO_TIME); // allows other to change
    }
}
```

```
void ChangeRoom::customer_3_change(void) {
    while (1) {
        if (door.trylock()) {
            cout << "Customer 3 wait: " << sc_time_stamp() << endl;
            door.lock();
        }
        cout << "Customer 3 start: " << sc_time_stamp() << endl;
        // carry out change
        wait(600, SC_SEC);
        cout << "Customer 3 end: " << sc_time_stamp() << endl;
        door.unlock();
        wait(SC_ZERO_TIME); // allows other to change
    }
}
```

```
void ChangeRoom::customer_1_change(void) {
    while (1) {
        if (door.trylock()) {
            cout << "Customer 1 wait: " << sc_time_stamp() << endl;
            door.lock();
        }
        cout << "Customer 1 start: " << sc_time_stamp() << endl;
        // carry out change
        wait(600, SC_SEC);
        cout << "Customer 1 end: " << sc_time_stamp() << endl;
        door.unlock();
        wait(SC_ZERO_TIME); // allows other to change
    }
}
```

```
ChangeRoom cr_i("cr_i");
sc_start(3000, SC_SEC);
```

```
Customer 1 start: 0 s
Customer 2 wait: 0 s
Customer 3 wait: 0 s
Customer 1 end: 600 s
Customer 2 start: 600 s
Customer 1 wait: 600 s
Customer 2 end: 1200 s
Customer 3 start: 1200 s
Customer 2 wait: 1200 s
Customer 3 end: 1800 s
Customer 1 start: 1800 s
Customer 3 wait: 1800 s
Customer 1 end: 2400 s
Customer 2 start: 2400 s
Customer 1 wait: 2400 s
```

Channels: sc_semaphore

- Allows multiple processes to share a pool of common resources without colliding.

- Declare: `sc_semaphore NAME(COUNT);`
- `sc_semaphore` channel implements `sc_semaphore_if` interface class
- Member functions
 - Lock (blocking, waits if not available): `wait()`
 - Lock (non-blocking, moves ahead if already locked): `trywait()`, returns 0 on success
 - Free a locked semaphore: `post()`
 - Get the number of available semaphore: `get_value()`

```
SC_MODULE(Parking) {
    sc_semaphore slots;
    void car_1(void); // process
    void car_2(void); // process
    void car_3(void); // process
    void car_4(void); // process
    SC_CTOR(Parking) : slots(2) {
        SC_THREAD(car_1); // register process
        SC_THREAD(car_2); // register process
        SC_THREAD(car_3); // register process
        SC_THREAD(car_4); // register process
    }
};
```

Semaphore: Example

```
void Parking::car_1(void) {  
    while (1) {  
        if (slots.trywait()) {  
            cout << "Car 1 wait: " << sc_time_stamp() << endl;  
            slots.wait();  
        }  
        cout << "Car 1 parked: " << sc_time_stamp() << endl;  
        // utilize parking slot  
        wait(100, SC_SEC);  
        cout << "Car 1 left: " << sc_time_stamp() << endl;  
        slots.post();  
        wait(SC_ZERO_TIME); // allows other to park  
    }  
}
```

- Similar threads for car_2, car_3, and car_4
- Assume utilization of:
 - Car_1: 100 s
 - Car_2: 200 s
 - Car_3: 300 s
 - Car_4: 400 s

```
Parking park_i("park_i");  
sc_start(1000, SC_SEC); //
```

```
Car 1 parked: 0 s  
Car 2 parked: 0 s  
Car 3 wait: 0 s  
Car 4 wait: 0 s  
Car 1 left: 100 s  
Car 3 parked: 100 s  
Car 1 wait: 100 s  
Car 2 left: 200 s  
Car 4 parked: 200 s  
Car 2 wait: 200 s  
Car 3 left: 400 s  
Car 1 parked: 400 s  
Car 3 wait: 400 s  
Car 1 left: 500 s  
Car 2 parked: 500 s  
Car 1 wait: 500 s  
Car 4 left: 600 s  
Car 3 parked: 600 s  
Car 4 wait: 600 s  
Car 2 left: 700 s  
Car 1 parked: 700 s  
Car 2 wait: 700 s  
Car 1 left: 800 s  
Car 4 parked: 800 s  
Car 1 wait: 800 s  
Car 3 left: 900 s  
Car 2 parked: 900 s  
Car 3 wait: 900 s
```

Channels: `sc_fifo<T>`

- FIFO: First-in first-out queues
- `sc_fifo<T>` class inherits from and implements two interface classes:
 - `sc_fifo_in_if<T>`: for reading
 - `sc_fifo_out_if<T>`: for writing

```
SC_MODULE(TicketCounter) {  
    sc_fifo<int> customers;  
    int token; // size=3  
    void add_new_customer(void);  
    void process_one_customer(void);  
    SC_CTOR(TicketCounter) : customers(3), token(1) {  
        SC_THREAD(add_new_customer);  
        SC_THREAD(process_one_customer);  
    }  
};
```

- Declare: `sc_fifo<ELEMENT_TYPENAME> NAME(SIZE)`
- Member functions
 - Writing (suspends if FIFO is full until data read event is notified): `write(VALUE)`
 - Non-blocking write: `nb_write(VALUE)`
 - Reading (suspends if FIFO is empty until data write event is notified): `read()`, `read(reference)`
 - Non-blocking read: `nb_read()`, `nb_read(reference)`
 - Number of available data for reading: `num_available()`
 - Number of free slots for writing: `num_free()`
 - `data_read_event()`, `data_write_event()`

Channels: sc_fifo<T>

```
void TicketCounter::add_new_customer(void) {
    while (1) {
        cout << "In queue: " << customers.num_available()
              << " at:" << sc_time_stamp() << endl;
        customers.write(token);
        cout << "Adding token: " << token << " at:"
              << sc_time_stamp() << endl;
        token++;
        wait(100, SC_SEC); // add ever 100 s
    }
}
```

```
void TicketCounter::process_one_customer(void) {
    while (1) {
        int token_number = customers.read();
        cout << "Processing: " << token_number << " at:"
              << sc_time_stamp() << endl;
        wait(600, SC_SEC); // processing takes 600 s
        cout << "Processed: " << token_number << " at:"
              << sc_time_stamp() << endl;
    }
}
```

```
TicketCounter tc_i("tc_i");
sc_start(1000, SC_SEC); //
```

```
In queue: 0 at:0 s
Adding token: 1 at:0 s
Processing: 1 at:0 s
In queue: 0 at:100 s
Adding token: 2 at:100 s
In queue: 1 at:200 s
Adding token: 3 at:200 s
In queue: 2 at:300 s
Adding token: 4 at:300 s
In queue: 3 at:400 s
Processed: 1 at:600 s
Processing: 2 at:600 s
Adding token: 5 at:600 s
In queue: 3 at:700 s
```

- FIFO size is 3 in the constructor (previous slide)
- FIFO becomes full at t=400
- add_new_customer waits for an empty slot

sc_clock

- Predefined channel to model clock
- Derived from `sc_signal`
- Constructor: name, period_time, period_time_unit, duty_cycle, ...

```
int sc_main(int sc_argc, char* sc_argv[]) {
    sc_clock myclk("myclk", 10, SC_NS, 0.6);
    Clock clk_i("clk_i");
    clk_i.clock(myclk);
    sc_start(21, SC_NS);
    return 0;
}
```

```
SC_MODULE(Clock) {
    void report(void); // process
    sc_in<bool> clock;
    SC_CTOR(Clock) {
        SC_THREAD(report);
        dont_initialize();
        sensitive << clock;
    }
};

void Clock::report(void) {
    while (1) {
        cout << name() << " clk:" << clock->read()
              << " T=" << sc_time_stamp() << endl;
        wait();
    }
}
```

```
clk_i clk:1 T=0 s
clk_i clk:0 T=6 ns
clk_i clk:1 T=10 ns
clk_i clk:0 T=16 ns
clk_i clk:1 T=20 ns
```

Transaction-level Models (TLM)

TLM: Problem of HDL Models

Hardware View

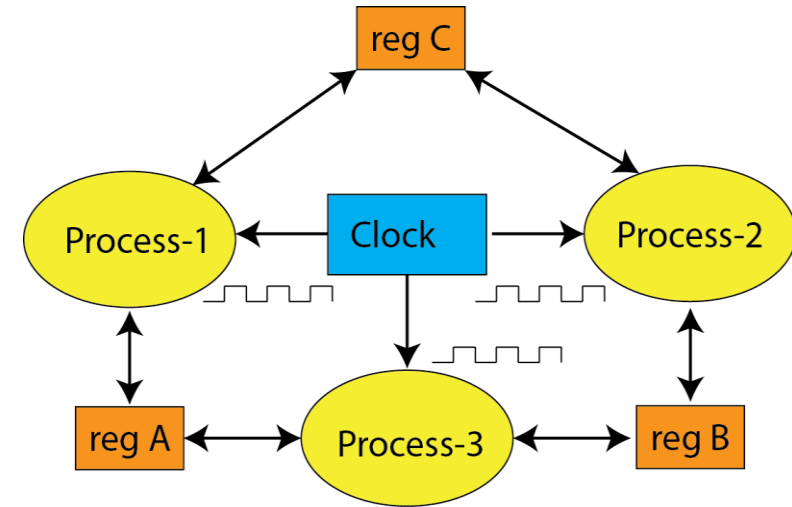
- All parallel components are active at the same time
- Modeled in Hardware Description Language (HDL) as processes
 - Verilog (always), VHDL (process)

- Processes active all the time
- Processes communicate via shared variables (wires, reg, signal)

Simulation:

- Captures cycle-based behavior
- Compute ALL process at each clock cycle

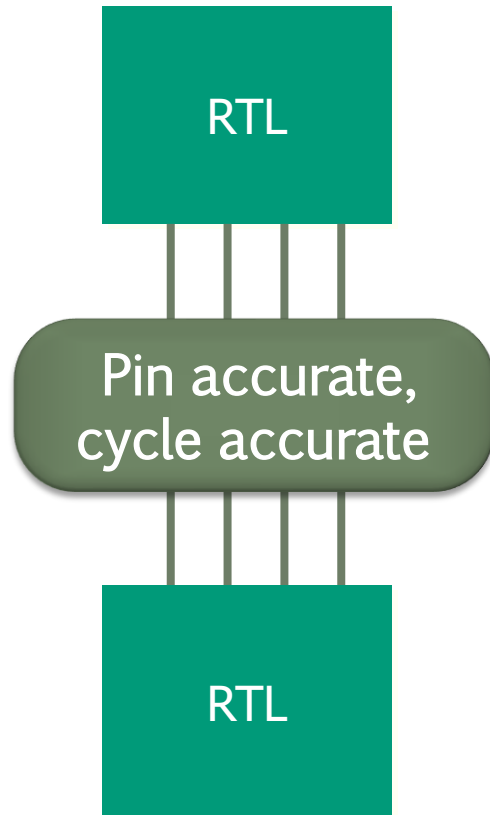
- TLM: compute only things/event of interest
 - Interest is defined by the type of analysis needed



Demerit: such detail is not required at ESL

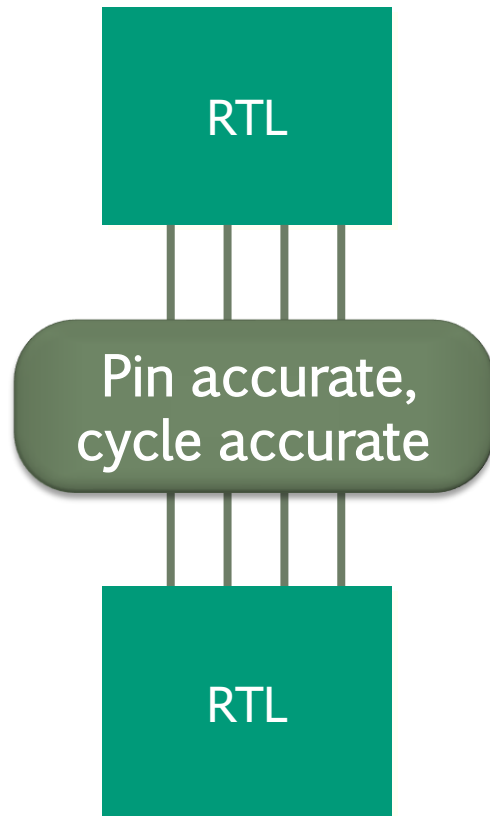
Solution: TLM provides a higher abstraction level for analysis

TLM: Why is it faster?

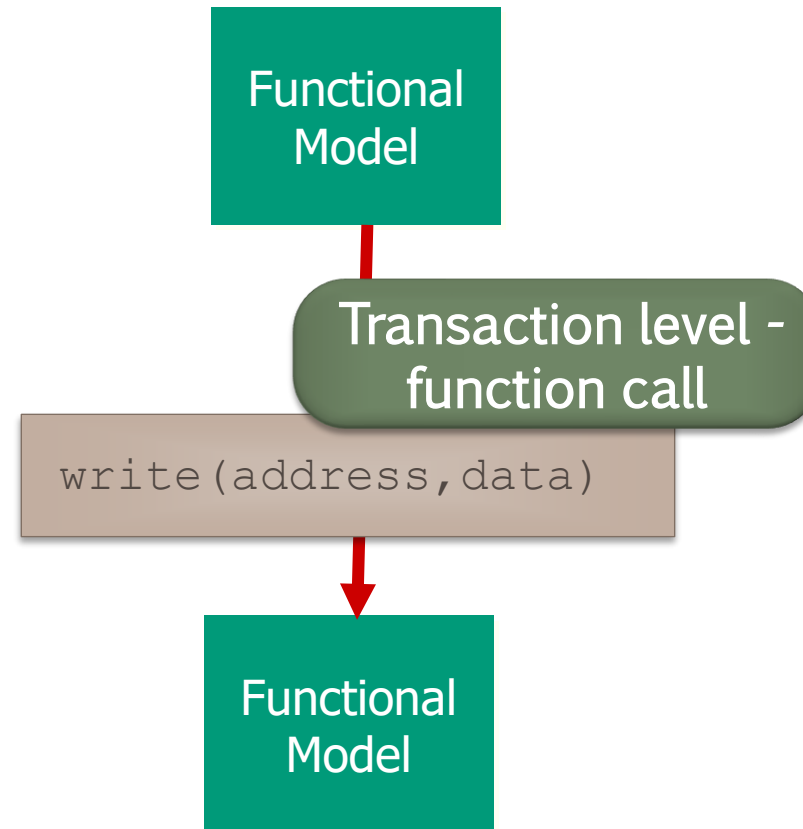


Simulate every event

TLM: Why is it faster?

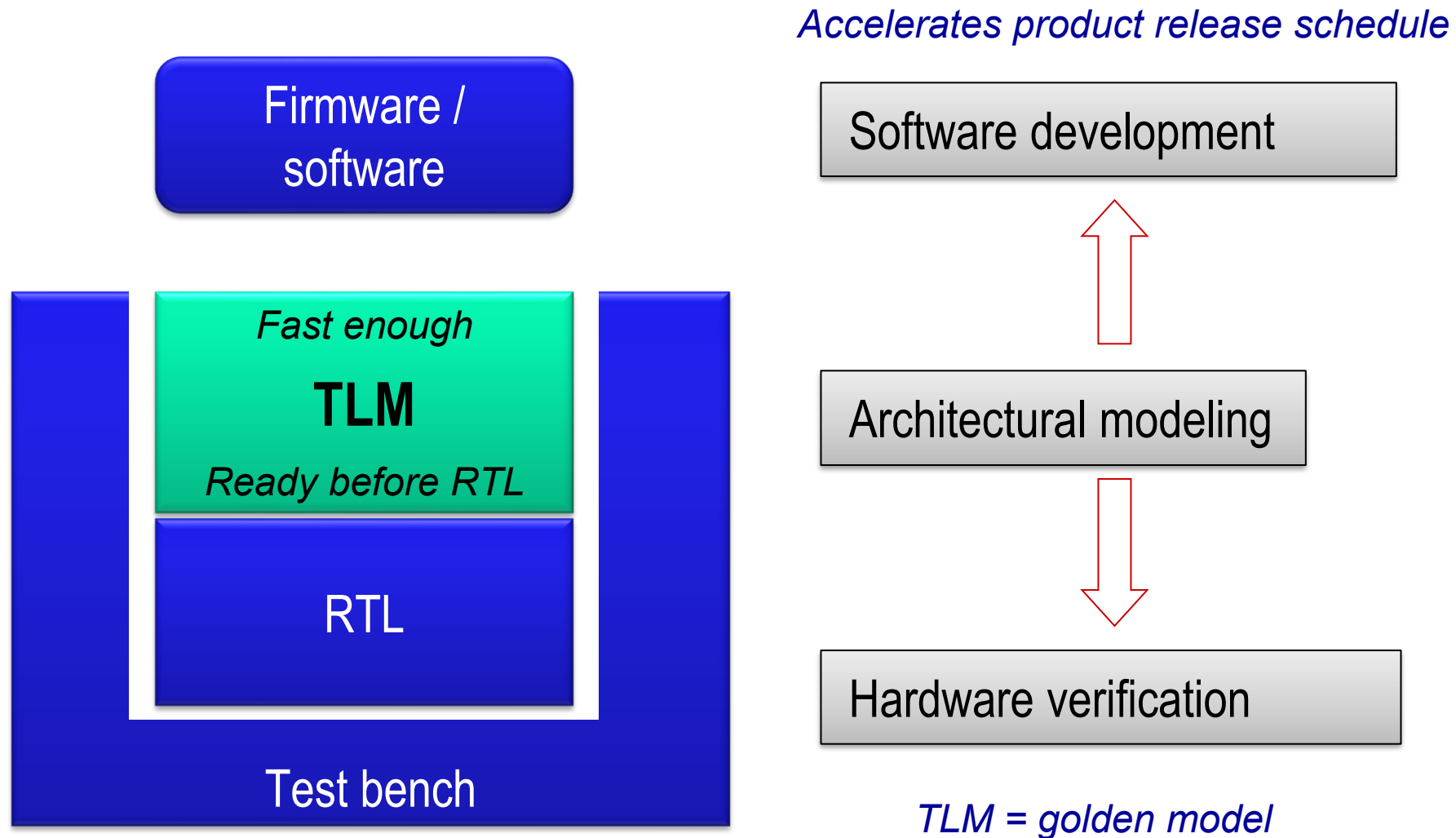


Simulate every event



100-10,000 X faster simulation

TLM: Applications



TLM: Basics

What is transaction?

- The individual components communicate by sending messages requesting data be transferred between each other.
- The exchange of messages is called a transaction

How is TLM different from RTL Model?

- Model each component only when it has something to do.
- This is the hardware behavior at the high level
 - Functional blocks communicate by reading and writing across buses.

TLM: Types of Timing Details

Loosely timed (LT):

- Uses transactions corresponding to a complete read or write across a bus or network in physical hardware.
- It provides timing at the level of the individual transaction.

Application (LT):

- Software application development.
- Software performance analysis.
- Hardware architectural analysis.
- Hardware functional verification.

Untimed (UT): less detail than LT

- Easily implemented as loosely timed models which always set the timing parameter in transport calls to zero.

Application (UT):

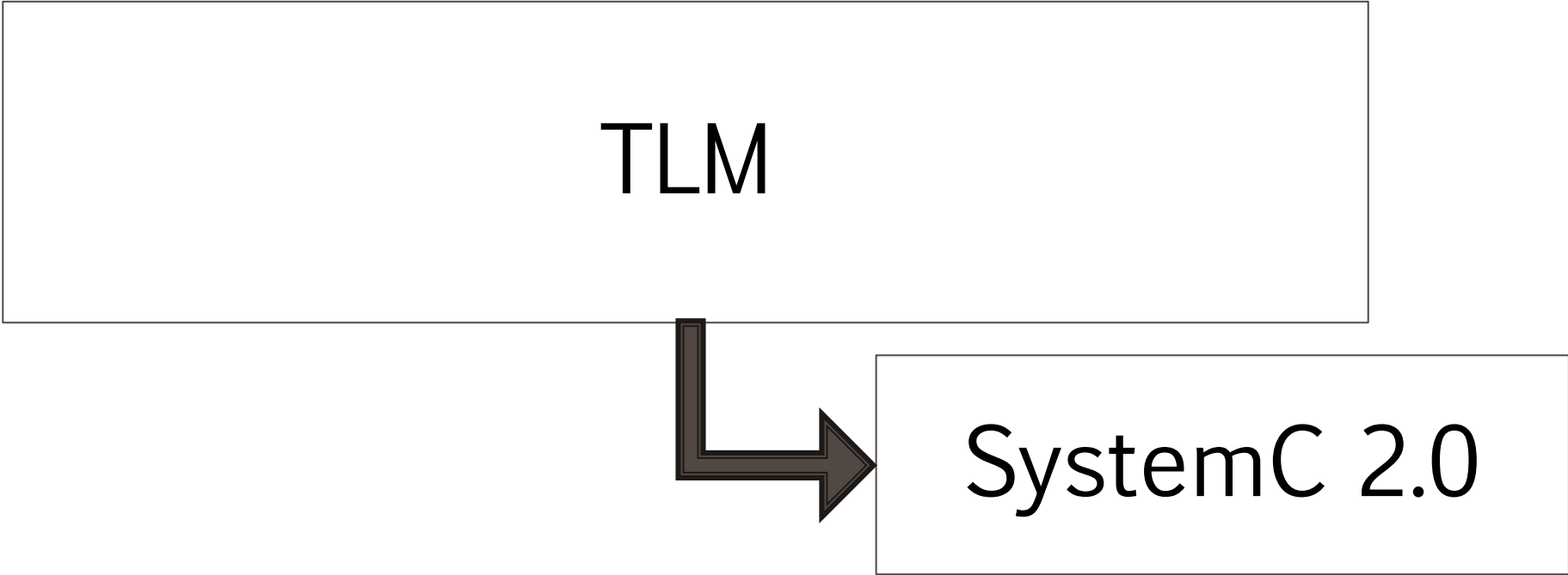
- Hardware functional verification.

Approximately timed (AT): lower than LT

- Breaks down transactions into a number of phases corresponding much more closely to the phasing of particular hardware protocols
- For example, the address and data phases of an AHB read or write.

Application (AT):

- Hardware architectural analysis.
- Hardware performance verification.
- Hardware functional verification.



A diagram consisting of two rectangular boxes. The top box is white with a black border and contains the text 'TLM'. The bottom box is also white with a black border and contains the text 'SystemC 2.0'. A large, dark grey arrow with a black outline points from the bottom center of the 'TLM' box to the left side of the 'SystemC 2.0' box. The arrow is L-shaped, pointing down and then right.

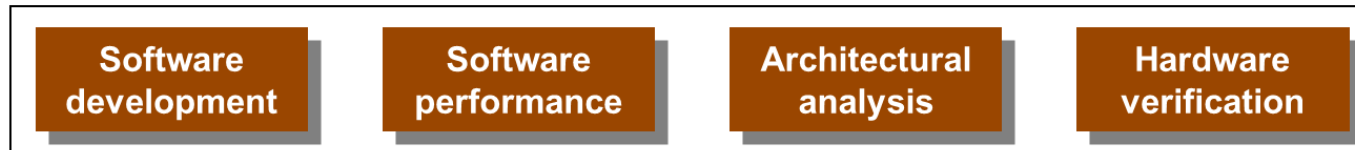
TLM

SystemC 2.0

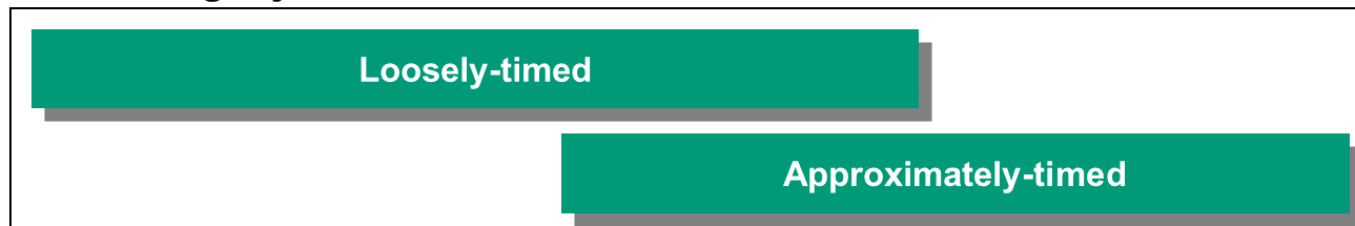
SystemC TLM: Overview

- Open SystemC Initiative (OSCI) working group on TLM proposed 1.0 in 2005 and 2.0 in 2008

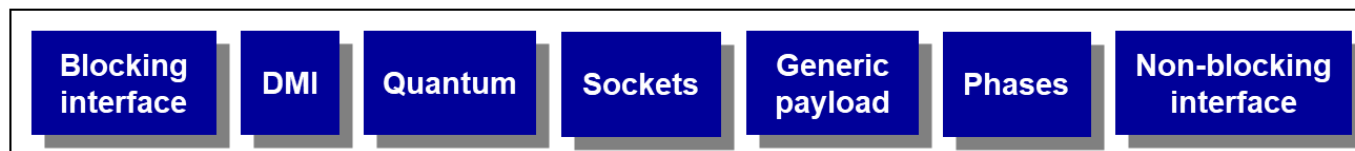
Use cases



TLM-2 Coding styles



Mechanisms



- Loosely-timed/Approximately-timed/Untimed
- Sockets
- Payload
- Blocking/Non-blocking Interfaces
- DMI
- Quantum

<https://systemc.org/overview/systemc-tlm/#resources>

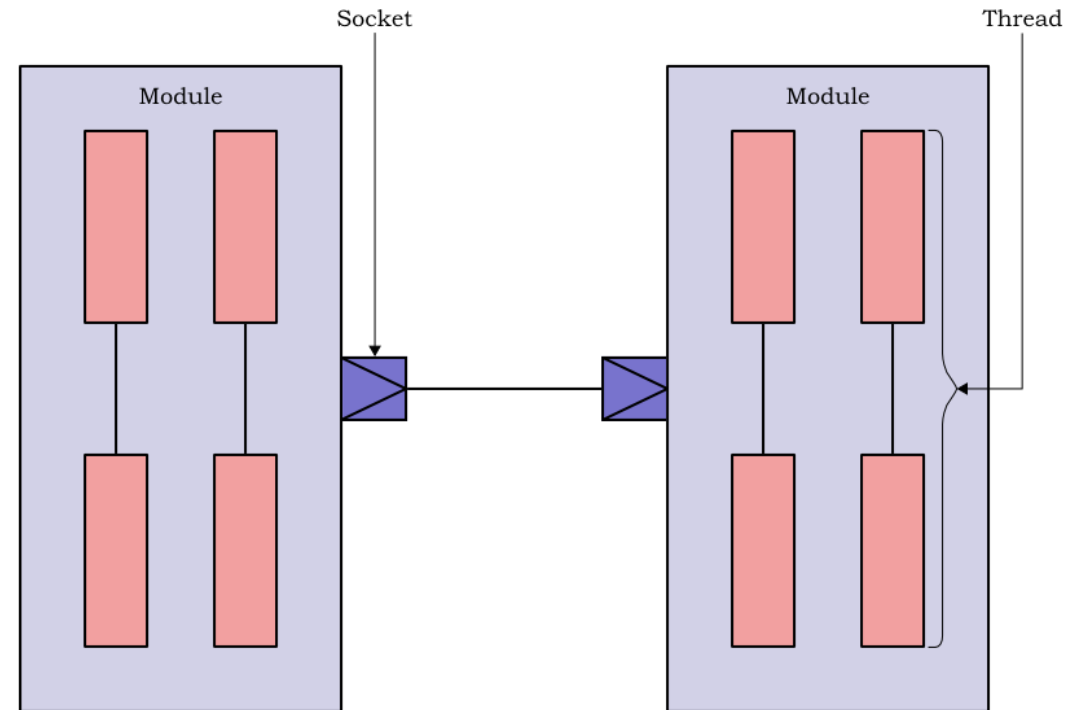
TLM: Sockets and Payload

- A TLM is a set of SystemC modules, each providing one or more sockets through which the SystemC modules may read and write data.

Socket: one endpoint of a two way communication link between two programs

- Module has processes
- Processes of one module communicate with the processes of other modules by passing data (i.e. reading or writing) through the sockets.

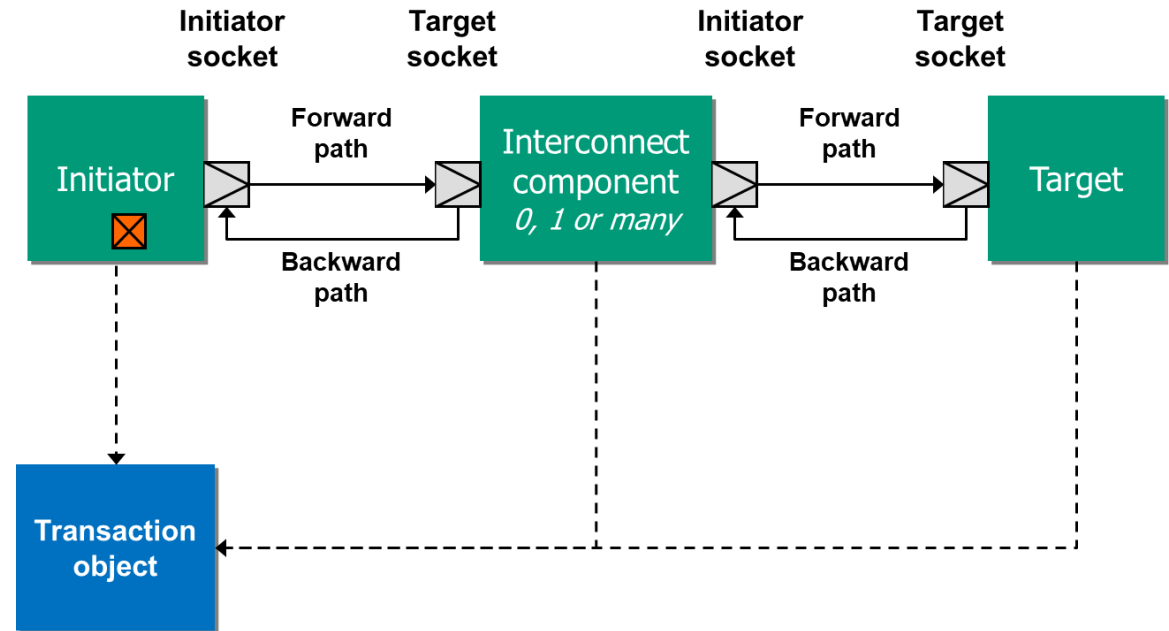
- This communication is known as a transaction
- The data passed is a **payload**.



<https://www.embecosm.com/appnotes/ean1/html/ch02s03.html>

TLM: Initiators and Targets

- A module's threads may act as either **initiators or targets**.
- **Initiator**: responsible for creating a payload and calling the transport function to send it.
- **Target**: receives payloads from the transport function for processing and produce response.
- Initiator calls are made through initiator sockets, target calls received through target sockets.
- A module may implement both target and initiator sockets, allowing its threads to both generate and receive traffic.



<https://www.accellera.org/downloads/standards/systemc/files>

References

- Black, David C., and Jack Donovan, eds. SystemC: From the ground up. Boston, MA: Springer US, 2004.